

中山大学计算机学院本科生实验报告

(2025 学年春季学期)

课程名称: 并行程序设计与算法
实验题目: Lab6 Pthreads 并行构造与 Heated Plate 并行应用
批改人:
专业 (方向):
学号: 23336128
姓名: 梁力航
Email:
完成日期: 2026 年 5 月 7 日

本次实验从课程给出的 `heated_plate_openmp.c` 出发, 保留原来的边界条件和 Jacobi 迭代公式, 只把内部网格的行循环换成自己写的 Pthreads `parallel_for`。为了观察任务划分方式的影响, 我另外做了 `block`、`cyclic`、`dynamic` 三种调度, 并把结果和 OpenMP 版本放在同一套脚本里比较。

1 实验目的

Heated Plate 本身的计算并不复杂，难点在于每一轮迭代都有固定的数据依赖：新矩阵只能从上一轮矩阵计算，不能直接原地更新。因此，本实验主要关注两件事：第一，怎样把内部网格点的更新拆成可并行的循环；第二，手写的 Pthreads `parallel_for` 和 OpenMP 版本相比，正确性和运行时间会有什么差别。

具体实验内容如下：

1. 理解 Heated Plate 稳态热传导问题的迭代过程；
2. 用 Pthreads 实现 `parallel_for`，并接入 Heated Plate；
3. 比较 `block`、`cyclic`、`dynamic` 三种调度方式；
4. 与课程资料中的 OpenMP 实现做正确性和运行时间对照；
5. 整理出可以复现实验的脚本、结果和报告。

2 实验平台与参考来源

表 1: 实验环境信息

项目	配置
宿主机	macOS / arm64
运行环境	Docker Linux (Ubuntu 24.04, arm64)
C++ 编译器	<code>g++</code> , <code>-O3 -std=c++17 -pthread -fopenmp</code>
并行模型	OpenMP / Pthreads
Python 工作流	<code>uv run python ...</code>
程序输出	<code>key=value</code> 形式，便于脚本解析

实验中用到的线程接口主要来自 POSIX Pthreads；计时部分 OpenMP 对照程序使用 `omp_get_wtime()`。相关接口参考如下：

- Pthreads 线程创建和等待参考 POSIX `pthread_create` / `pthread_join`: http://pubs.opengroup.org/onlinepubs/9699919799/functions/pthread_create.html, https://pubs.opengroup.org/onlinepubs/9699919799/functions/pthread_join.html。

- Pthreads 动态调度中的共享游标使用互斥锁保护, 参考 POSIX `pthread_mutex_lock`: https://pubs.opengroup.org/onlinepubs/9699919799/functions/pthread_mutex_lock.html。
- OpenMP 对照程序用 `omp_get_wtime()` 统计 wall-clock 时间, 参考 OpenMP 5.0 文档: <https://www.openmp.org/spec-html/5.0/openmps160.html>。

3 问题描述与并行化思路

Heated Plate 使用二维规则网格描述矩形区域的温度。边界条件为: 上边界温度为 0, 左、右和下边界温度为 100。内部点每轮迭代按照四邻域平均值更新:

$$W_{i,j}^{new} = \frac{1}{4}(U_{i-1,j} + U_{i+1,j} + U_{i,j-1} + U_{i,j+1})$$

当相邻两轮之间的最大变化量 `diff` 小于给定阈值 `epsilon` 时停止。

我采用的是 Jacobi 更新方式。每轮先把当前矩阵 `W` 复制到 `U`, 再根据 `U` 算出新的 `W`。这样做虽然多了一次复制, 但好处是同一轮里每个内部点只读旧矩阵, 写入的位置也互不重叠。因此可以把内部行交给不同线程处理, 不需要在更新矩阵时加锁。

4 核心实现

4.1 `parallel_for` 调度方式

在 Lab5 的连续块划分基础上, 我这里保留并扩展了三种调度:

- `block`: 将迭代空间按连续块均分给各线程;
- `cyclic`: 每个线程按固定步长轮转领取 `chunk`;
- `dynamic`: 多个线程通过互斥锁保护的共享游标动态领取 `chunk`。

其中 `dynamic` 的主要代码如下。它的思路是用一个共享游标记录下一个还没分配的 `chunk`, 每个线程取任务时先加锁, 取完后马上释放锁:

```
pthread_mutex_lock(&shared->mutex);
iter_begin = shared->next_iter;
shared->next_iter += shared->chunk_size;
pthread_mutex_unlock(&shared->mutex);

if (iter_begin >= shared->total_iters) {
    break;
```

```
}  
iter_end = std::min(iter_begin + shared->chunk_size, shared->  
    total_iters);  
execute_range(shared, iter_begin, iter_end);
```

4.2 Heated Plate 并行化

Pthreads 版本每一轮迭代拆成三个并行循环：

1. 复制矩阵： $W \rightarrow U$;
2. 更新内部行：根据 U 计算新的 W ;
3. 计算每一行的局部最大变化量，并由主线程规约得到全局 diff 。

计算 diff 时，我没有让多个线程直接更新同一个全局变量，而是先把每一行的局部最大变化量写到 $\text{row_diff}[i]$ ，最后由主线程顺序扫描。这种写法比较朴素，但结果更容易检查，三种 Pthreads 调度和 OpenMP 对照也能得到相同的 checksum。

5 实验结果

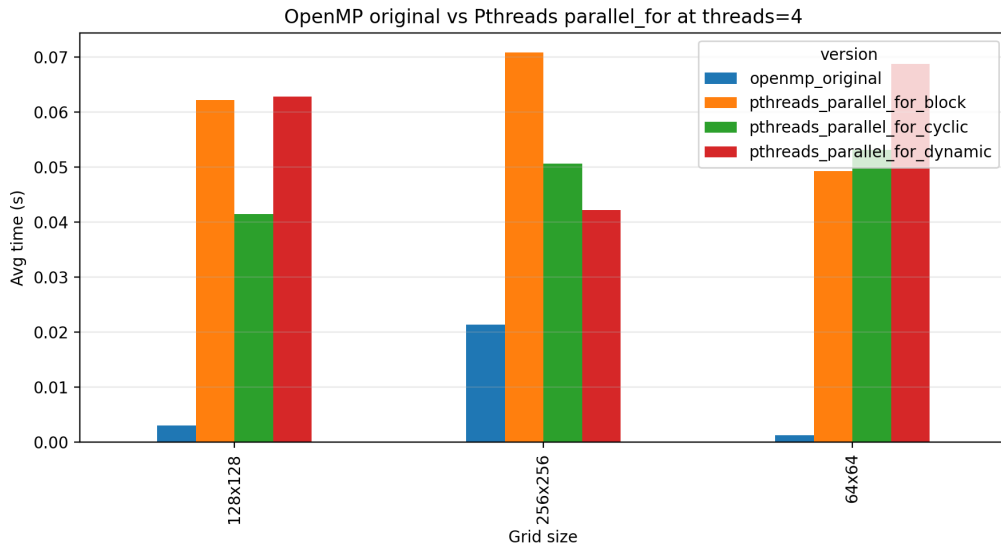
我在仓库中保留了一组中等规模的测试结果，配置如下：

- 网格规模： 64×64 、 128×128 、 256×256 ;
- 线程数：1、2、4;
- 阈值： $\text{epsilon}=0.1$;
- Pthreads chunk 大小：8;
- 每组运行次数：1。

完整数据保存在 `results/benchmark_results.json`、`results/summary_by_version.c` 和 `results/performance_table.csv`。这组数据一共 36 条记录，状态均为 ok；Pthreads 三种调度方式和 OpenMP 对照的 checksum 也保持一致。

表 2: 部分运行时间结果 (秒)

后端	版本 / 调度	线程数	128×128	256×256
OpenMP	<code>openmp_original</code>	1	0.012308	0.019530
OpenMP	<code>openmp_original</code>	2	0.001909	0.008796
Pthreads	<code>block</code>	1	0.028204	0.045497
Pthreads	<code>block</code>	2	0.024976	0.029151
Pthreads	<code>cyclic</code>	2	0.030628	0.031942
Pthreads	<code>dynamic</code>	2	0.033584	0.055894

图 1: OpenMP 原始实现与 Pthreads `parallel_for` 版本对比

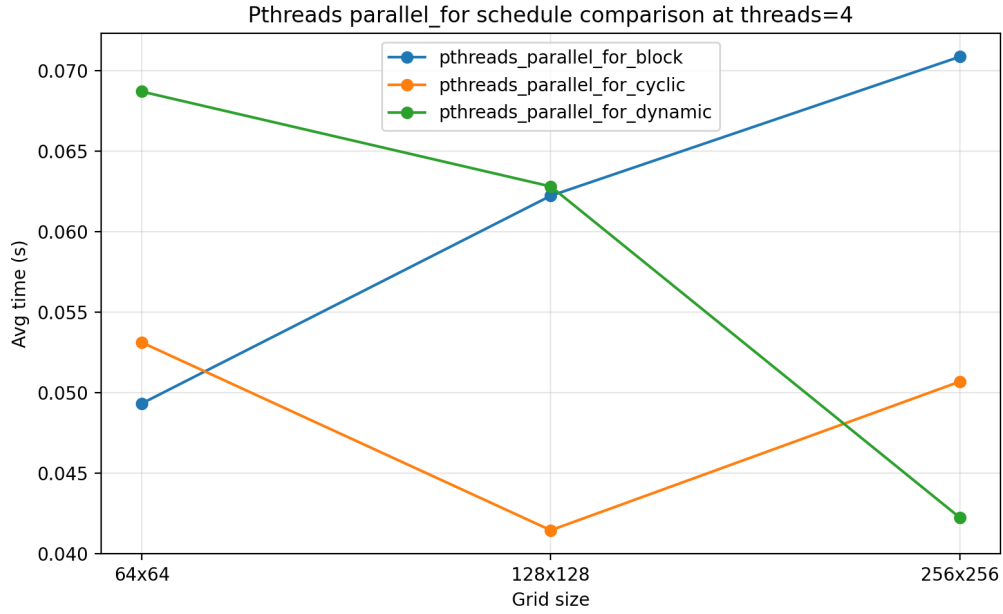
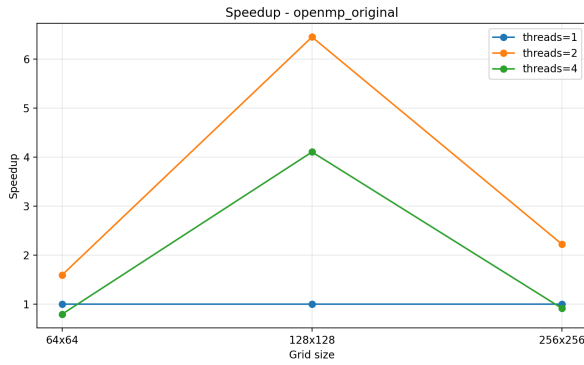
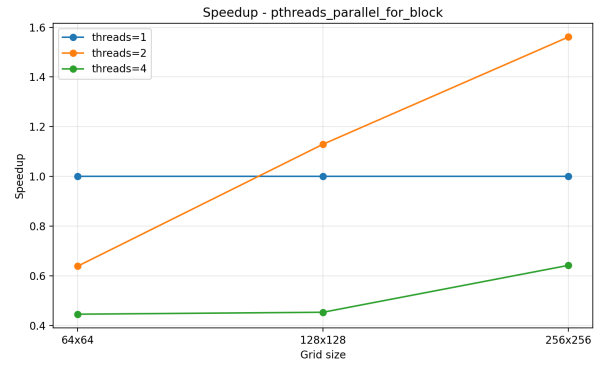


图 2: Pthreads parallel_for 三种调度方式对比



(a) OpenMP speedup



(b) Pthreads block speedup

图 3: 加速比曲线示例

6 结果分析

从表格和图中可以看到，OpenMP 版本整体明显更快。这个结果并不意外。两边的数值计算公式是一样的，差距主要来自运行时开销：我写的 `parallel_for` 每调用一次都会创建一批 Pthreads 线程，执行完再回收；而 Heated Plate 每轮迭代里至少要并行执行复制、更新和计算 `diff` 三段循环，线程管理开销被重复放大了。

三种 Pthreads 调度之间，`block` 在这个问题上反而更稳。原因是规则网格里每一行的计算量基本相同，连续分块已经能把任务分得比较均匀。`dynamic` 的优势一般出现在任务不均衡时，但这里每次取 `chunk` 都要访问互斥锁，额外开销抵消了它的灵活性。`cyclic` 可以把行号打散，但对本实验这种规则访问模式来说，没有明显收益。

7 验证

为了避免只看运行时间而忽略正确性，我写了两组测试：

- `tests/test_core_programs.py`: 验证 OpenMP 与三种 Pthreads 调度的 checksum 和迭代次数一致，验证 `--dump` 输出和动态库链接；
- `tests/test_benchmark.py`: 用缩小配置跑通 benchmark、plot 和 export 流程，检查 JSON、CSV、图表和表格产物。

本次提交前执行了：

```
uv run python -m unittest ./tests/test_core_programs.py
uv run python -m unittest ./tests/test_benchmark.py
```

两组测试均通过。额外检查了 benchmark 结果文件，其中 36 条记录均为 ok。

8 结论

本次实验完成了 Heated Plate 从 OpenMP 参考程序到 Pthreads `parallel_for` 版本的改造。三种调度方式都能得到和 OpenMP 对照一致的迭代次数与 checksum，说明并行拆分没有破坏 Jacobi 迭代的计算逻辑。

性能方面，手写 `parallel_for` 版本没有超过 OpenMP。这个结果也说明了一个问题：仅仅把循环拆给多个线程并不等于一定更快，线程创建、同步和任务划分本身也会消耗时间。如果继续优化，我会优先把 `parallel_for` 改成持久线程池，让多个迭代阶段复用同一批线程，而不是每次进入并行循环都重新创建线程。